

Unit IV: File Handling and Dictionaries

Section 1: File Handling in Python

Python allows you to **work with files** (read, write, update) to store and retrieve data.

1.1 Introduction to Files

- A **file** is a collection of information stored on a **permanent storage device**.
- Python can handle **text files** and **binary files**.

File Types:

1. **Text files** (.txt, .csv, .log) → readable by humans
 2. **Binary files** (.jpg, .png, .exe) → stored in machine-readable format
-

1.2 File Path

- **Path** tells Python **where the file is located**.
 - Types of paths:
 - **Absolute path:** Complete path from root
 - "C:/Users/Name/Documents/file.txt"
 - **Relative path:** Relative to current working directory
 - "file.txt"
-

1.3 Opening and Closing Files

- Use `open()` to **open a file** and `close()` to **close it**.
- **Syntax:**

```
file = open("filename.txt", "mode")  
file.close()
```

File Modes:

Mode	Description
'r'	Read (default)

Mode	Description
'w'	Write (creates new file or overwrites)
'a'	Append (add data at end)
'rb'	Read binary
'wb'	Write binary

Example:

```
# Writing to a file
file = open("example.txt", "w")
file.write("Hello, Python!")
file.close()
```

1.4 Reading and Writing Files

- **Writing:** Use `write()` method

```
with open("file.txt", "w") as f:
    f.write("Python File Handling")
```

- **Reading:** Use `read()`, `readline()`, or `readlines()`

```
with open("file.txt", "r") as f:
    content = f.read()
print(content)
```

Example:

```
with open("data.txt", "w") as f:
    f.write("Line 1\nLine 2\nLine 3")

with open("data.txt", "r") as f:
    print(f.read())
```

1.5 File Positions

- Python keeps a **file pointer** to know where to read/write.
- Methods:
 - `tell()` → Returns current pointer position
 - `seek(offset)` → Moves pointer to specified position

```
with open("data.txt", "r") as f:
    print(f.read(5)) # Reads first 5 characters
    print(f.tell()) # Current position
    f.seek(0)       # Move to beginning
```

1.6 Renaming and Deleting Files

- Use `os` module for file operations.

```
import os

# Rename file
os.rename("old.txt", "new.txt")

# Delete file
os.remove("new.txt")
```

1.7 Directory Methods

- Python can **manage folders/directories**.
- Using `os` module:

```
import os

# Current working directory
print(os.getcwd())

# List all files/folders
print(os.listdir())

# Create a new directory
os.mkdir("new_folder")

# Remove a directory
os.rmdir("new_folder")
```

Section 2: Dictionaries in Python

Dictionaries are **key-value pairs** that allow **fast data lookup and modification**.

2.1 Creating Dictionaries

- Syntax:

```
student = {"name": "Alice", "age": 20, "grade": "A"}
```

- Can also use `dict()` constructor:

```
student = dict(name="Alice", age=20)
```

2.2 Accessing Values

- Access value using **key**:

```
print(student["name"]) # Alice
```

- Use `get()` to avoid errors if key not found:

```
print(student.get("city", "Not Found")) # Not Found
```

2.3 Adding and Updating Values

- Add new key-value pair:

```
student["city"] = "Pune"
```

- Update existing value:

```
student["age"] = 21
```

2.4 Common Dictionary Methods

Method	Description
<code>keys()</code>	Returns all keys
<code>values()</code>	Returns all values
<code>items()</code>	Returns all key-value pairs
<code>pop(key)</code>	Removes a key-value pair
<code>update(dict2)</code>	Updates dictionary with another dictionary

Example:

```
student = {"name": "Alice", "age": 20}
student["grade"] = "A"
student.update({"city": "Mumbai"})
print(student)
# Output: {'name': 'Alice', 'age': 20, 'grade': 'A', 'city': 'Mumbai'}
```

Summary for Students

- **File Handling:** Allows reading/writing/updating files, handling file positions, and managing directories.
- **Dictionaries:** Provide **fast, flexible data storage** with key-value pairs, suitable for storing structured information.

- **Modules Required:** `os` for file and directory management.